EDUHUB / ENGINEERING DOCUMENTATION

DEPLOYMENT DIAGRAM

EduHub | Online Learning Platform

01

HOSTING

ASP.NET Core MVC on Azure App Service

02

DATA & MEDIA

Azure SQL / Blob Storage / secure streaming

03

OPERATIONS

Stripe, edge delivery, telemetry and CI/CD

DEPLOYMENT MODEL

One deployable N-Tier modular monolith, managed Azure data services, and an external Stripe payment boundary. Required technologies and proposed infrastructure additions are explicitly separated.

SOURCE EduHub five-page brief + established technology/architecture baseline

SYSTEM ANALYSIS & DESIGN / 05 NOVEMBER 2026

01 | Deployment decision baseline

Purpose, scope and alignment with the approved EduHub requirements.

WHAT THIS ARTIFACT ANSWERS

Where EduHub executes in production, which deployable artifacts are installed on each runtime node, how browsers reach services, and how the platform communicates with Azure SQL, private media storage, Stripe and operational tooling.

ARCHITECTURE POSITION

One application deployment, three logical projects. EduHub.Web, EduHub.Services and EduHub.Data are compiled and deployed together in an ASP.NET Core MVC App Service. Azure SQL, Azure Blob Storage and Stripe remain separate managed/external resources. [S1, S2, R1]

EVIDENCE AND SCOPE LABELS

STATUS	MEANING	APPLICATION TO DIAGRAMS
REQUIRED	Explicit project-brief requirement	App Service, MVC, Azure Blob, Stripe, CDN concept, Identity, EF Core.
ESTABLISHED	Previously adopted EduHub design baseline	Single deployable N-Tier modular monolith; SQL Server / Azure SQL physical model.
RECOMMENDED	Proposed cloud infrastructure detail	Front Door Standard/Premium, Key Vault, Insights, managed identity, deployment slots.
OPTIONAL	Future hardening or scaling	SQL private endpoint, VNet outbound integration, dedicated media worker.

Out of scope: full API contracts, deployment scripts, cost calculations, detailed component internal APIs, and executable infrastructure-as-code. The deployment layout is a design proposal, not an assertion that Azure resources already exist.

02 | UML deployment notation

Correctly distinguish runtime nodes, artifacts and network relationships.

NOTATION AND INTERPRETATION

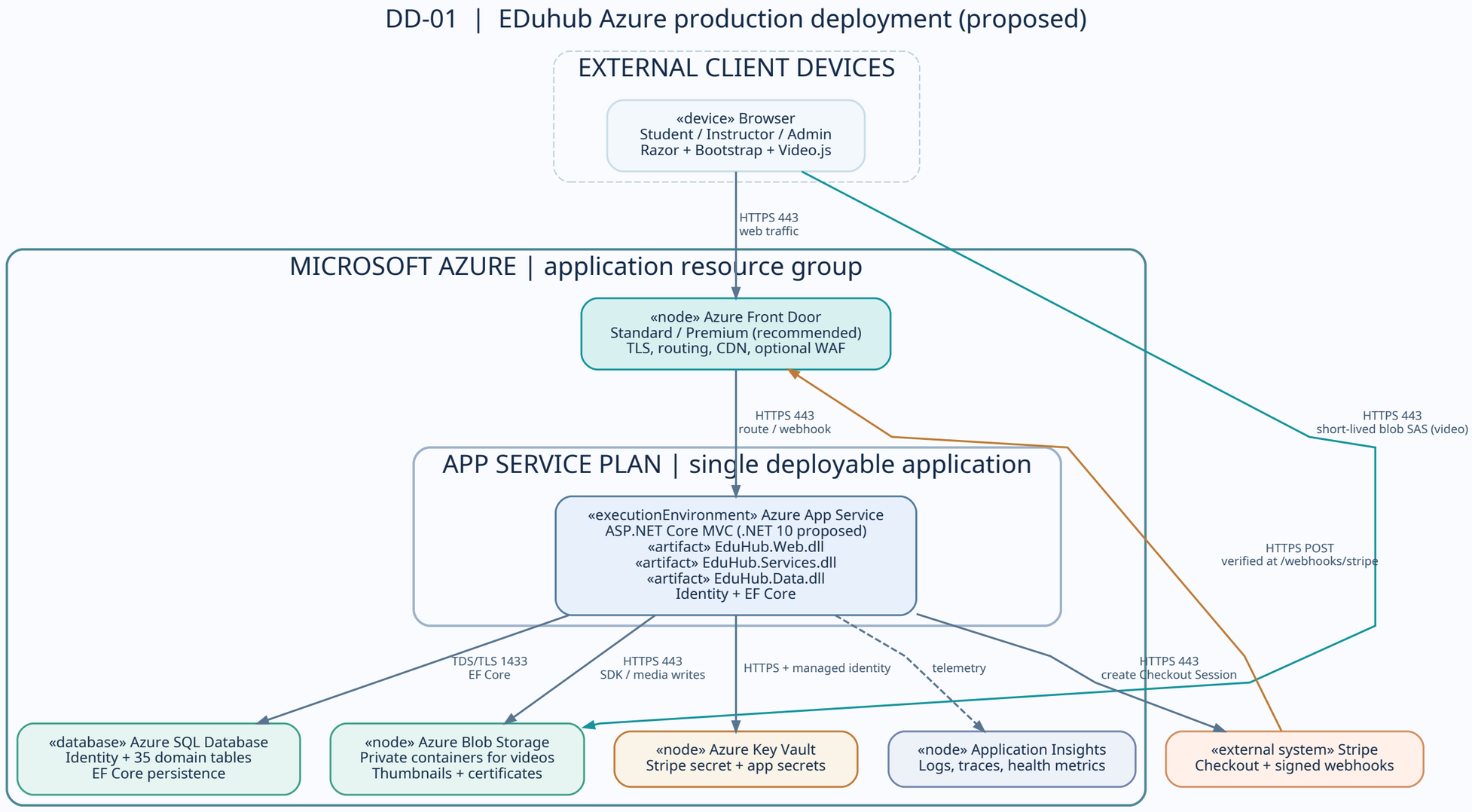
UML ELEMENT	EDUHUB EXAMPLE	HOW TO READ IT
«device»	Student / instructor browser	Client machine or browser execution endpoint.
«executionEnvironment»	Azure App Service / ASP.NET Core	Managed environment where compiled EduHub code executes.
«artifact»	EduHub.Web.dll, Services.dll, Data.dll	Physically deployed .NET output, not independent Azure services.
«database»	Azure SQL Database	Managed persistent relational database; outside the app process.
«node»	Azure Front Door, Blob Storage, Key Vault	External infrastructure service participating in deployment.
Communication path	HTTPS 443; TDS/TLS 1433	Actual connection protocol; a dependency arrow alone is not physical placement.
Dashed / optional	Worker; hardened private networking	A proposed option, not part of the baseline runtime.

WHY LAYERS ARE NOT SEPARATE CLOUD MACHINES

The course brief uses an N-Tier design with Web, Services and Data projects. These are logical code responsibilities. In the selected modular monolith, the assemblies share one host process; separate virtual machines, containers, Kubernetes clusters or microservice APIs are not required. [S1, S2, R1]

03 | Core UML deployment diagram

The proposed production node-and-artifact topology for a first deployable release.



READING THE DIAGRAM

The outer nodes identify separately hosted systems; the nested application boundary shows that the three project assemblies share a single App Service runtime. Solid arrows mark network communication. Azure Front Door and observability are recommended additions.

IMPORTANT: Azure CDN is a brief requirement; Front Door is the proposed modern edge implementation.

04 | Runtime deployment inventory

Deployable artifacts, hosting responsibilities and non-deployed dependencies.

PRIMARY DEPLOYMENT UNITS

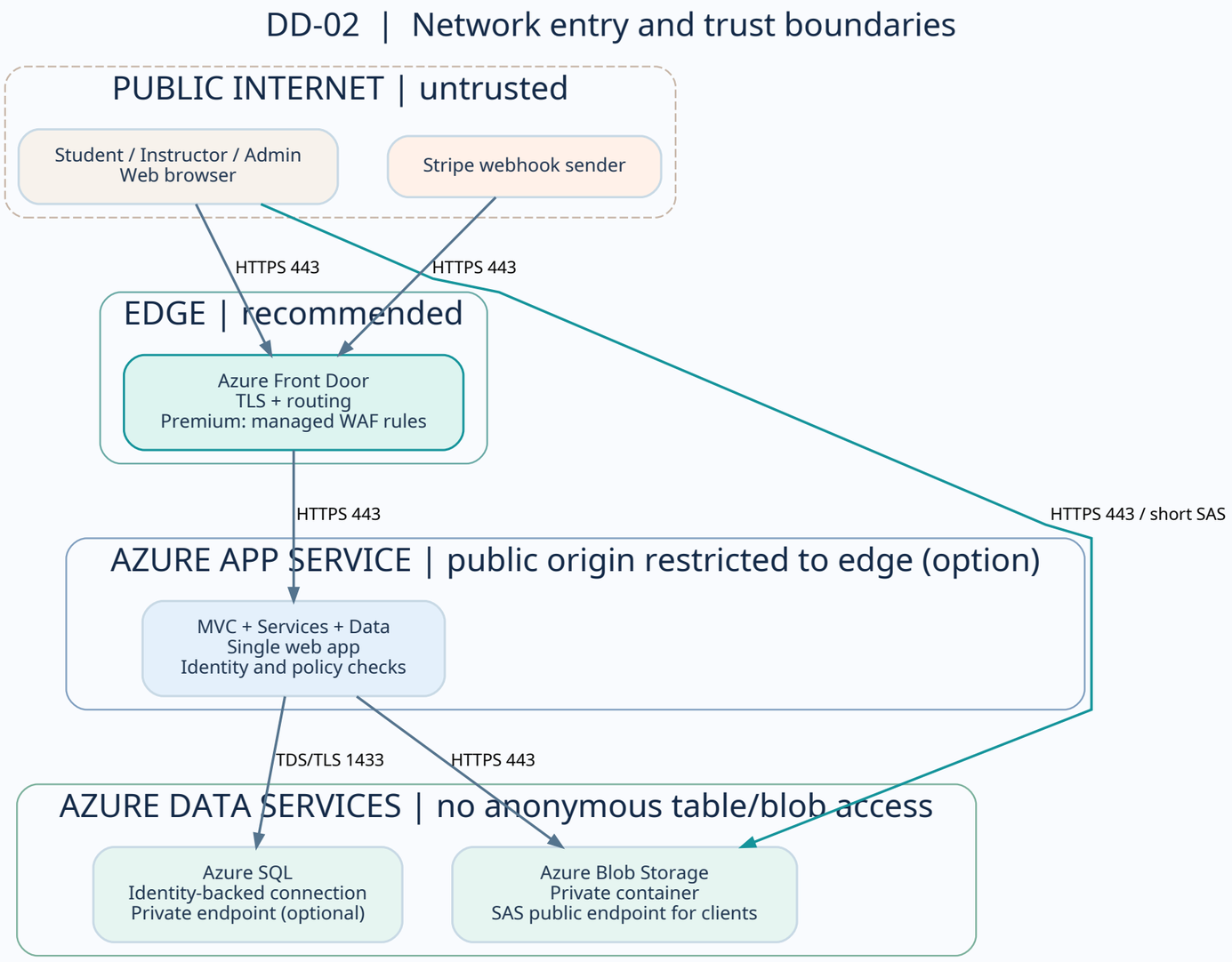
NODE / RUNTIME	DEPLOYED / OWNED CONTENT	PROVISIONING
Azure App Service	EduHub.Web.dll + Services.dll + Data.dll; Razor views, static assets, ASP.NET Identity middleware; EF Core runtime	Required Azure host; .NET 10 LTS is proposed [S1, S2].
Azure SQL Database	ASP.NET Identity tables; 35 custom EduHub domain tables; migration history	Established Azure SQL recommendation; SQL Server dialect.
Azure Blob Storage	Private course videos; thumbnails; course assets; generated certificates	Required managed storage.
Azure Front Door	Ingress routing, optional managed WAF rules, edge caching for safe public content	Recommended CDN replacement/implementation [R2].
Stripe SaaS	Hosted Checkout and payment-event delivery	Required external provider; never hosted inside Azure subscription.
Key Vault + Insights	Payment webhook secret and diagnostics	Recommended managed operational services.
CI runner / Git	Versioned code; build, test and release artifacts	Recommended development-side infrastructure.

PHYSICAL PLACEMENT CHECK

The App Service contains business logic and data-access code, but not the database files. SQL tables live in Azure SQL. Video bytes live in Blob Storage. Stripe remains a separately operated external payment service.

05 | Network ingress and trust

External traffic, primary request hops and the boundary around sensitive data.



READING THE DIAGRAM

Browsers and Stripe enter through HTTPS. Web requests terminate at the public application edge and are validated inside App Service. Video.js receives an authorized, short-lived Blob URL only after a server-side access check.

NOTE: App Service VNet integration is outbound-only; it does not make the web app privately accessible.

06 | Communication contract

Network direction, authentication and permitted business purpose.

CONNECTION CATALOG

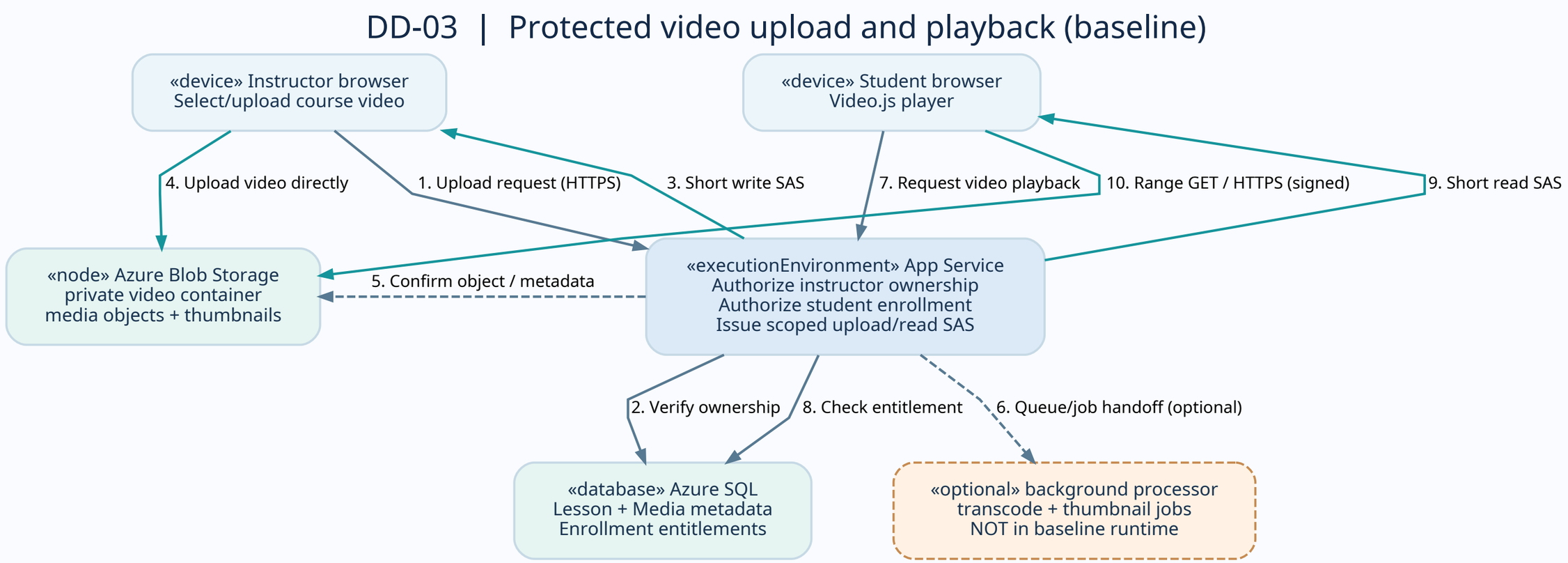
FROM -> TO	PROTOCOL / PORT	PURPOSE & SECURITY CONDITION
Browser -> Front Door / App	HTTPS 443	MVC views and authenticated commands; cookies and antiforgery.
Stripe -> application webhook	HTTPS 443	Public callback; raw-body Stripe signature verification; idempotent event processing.
App -> Azure SQL	TDS/TLS 1433	EF Core queries and transactional writes. Managed identity preferred; restrict database ingress.
App -> Azure Blob	HTTPS 443	Upload metadata and media-management operations using managed identity.
Browser -> Blob media endpoint	HTTPS 443	Range GET for signed short-duration Blob SAS; private container, no anonymous public listing.
App -> Stripe API	HTTPS 443	Create Checkout Session, retrieve authoritative payment status as needed.
App -> Key Vault / telemetry	HTTPS 443	Retrieve secrets via managed identity; emit traces without credentials or SAS tokens.

SECURITY NOTE

A private Blob container and a publicly reachable storage endpoint are compatible: access to objects requires a valid signed authorization token. Making the entire Blob endpoint private-only would break direct student playback from the public internet; a separate authenticated edge-delivery design would then be required. [R3, R6]

07 | Media deployment path

Instructor upload and learner playback without routing large video payloads through MVC.



READING THE DIAGRAM

The backend authorizes upload ownership and student enrollment, issues short-lived scoped SAS tokens, and persists metadata in SQL. The browser transfers video bytes directly to Blob Storage. The optional worker is shown as a future processing addition.

SECURITY: no public blob containers; avoid shared CDN caching of signed, per-user media URLs.

08 | Media delivery rules

Concrete storage and playback deployment decisions.

UPLOAD, PROCESSING AND PLAYBACK

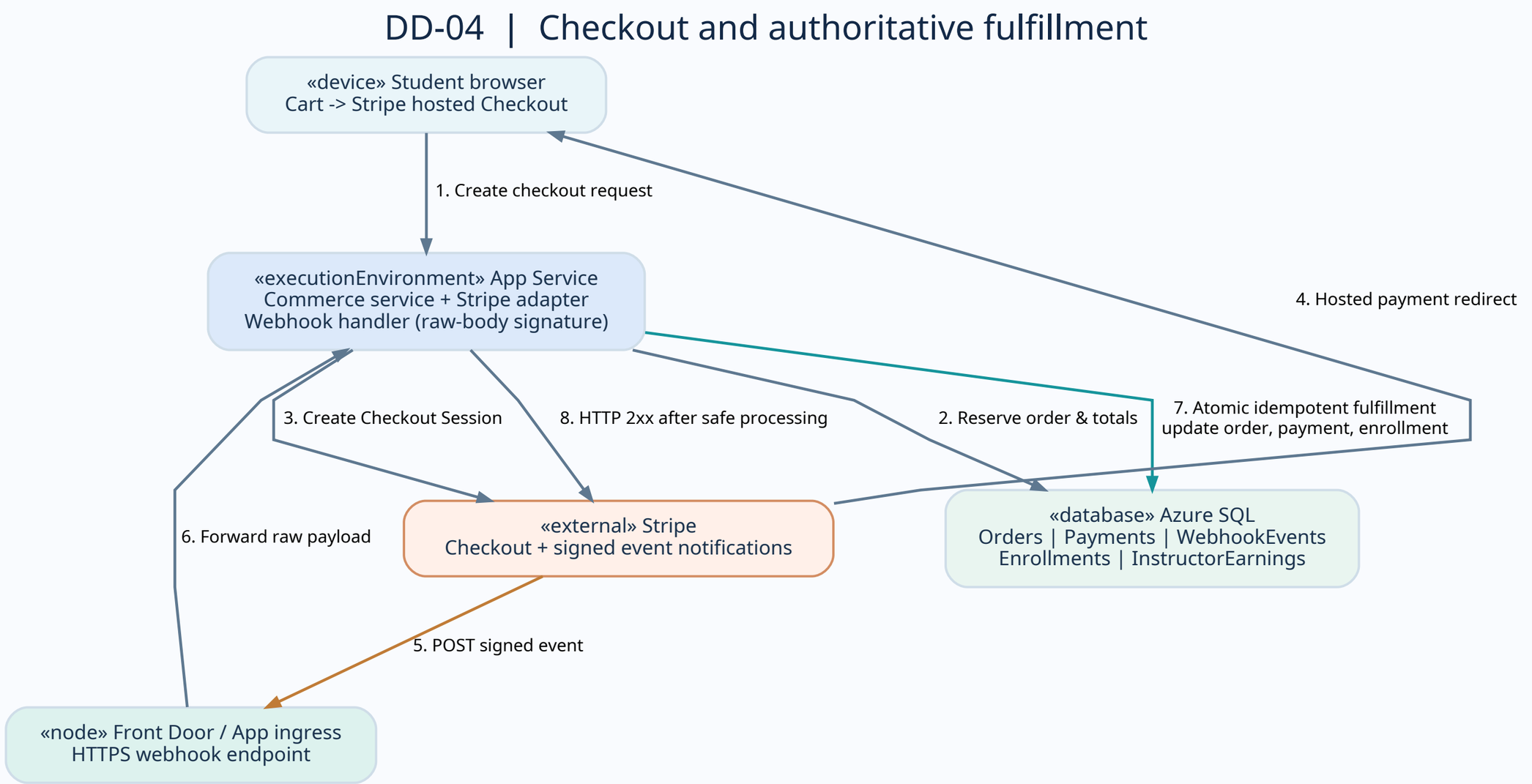
STAGE	RUNTIME RESPONSIBILITY	STATE / RISK CONTROL
1. Instructor upload	MVC API verifies ownership; creates scoped upload authorization.	Upload expiry, maximum size, allowed type and blob key validation.
2. Video transfer	Browser -> Blob HTTPS; backend stores asset metadata.	Do not pass media bytes through MVC by default.
3. Processing	Compression/thumbnail step can be a job or optional worker.	Processing compute service is not specified in brief; confirm implementation.
4. Student request	MVC checks enrollment, course publication, lesson visibility.	Reject unauthorized access before issuing signed playback URL.
5. Video.js playback	Browser streams via HTTP Range requests to media origin.	Use read-only, short-lived user-delegation SAS where feasible [R3].
6. CDN policy	Public thumbnails/assets can be cached at the edge.	Do not assume protected videos are safe to cache by default [R6].

CDN DECISION GATE

The original brief requests Azure CDN. The recommended current product is Azure Front Door Standard/Premium for edge distribution. Its video authorization and query-string caching rules must be decided before protected media is routed through the CDN. [S1, R2, R6]

09 | Commerce integration topology

Stripe Checkout, server-owned order state, and verified webhook fulfillment.



READING THE DIAGRAM

The browser initiates Checkout but cannot itself grant course access. Stripe sends a signed HTTPS callback; the app verifies it, deduplicates the event, and atomically updates payment and enrollment records in Azure SQL.

RULE: successful redirect is not proof of payment; verify the authoritative payment state.

10 | Commerce deployment rules

A reliable payment integration depends on a reachable, verifiable server endpoint.

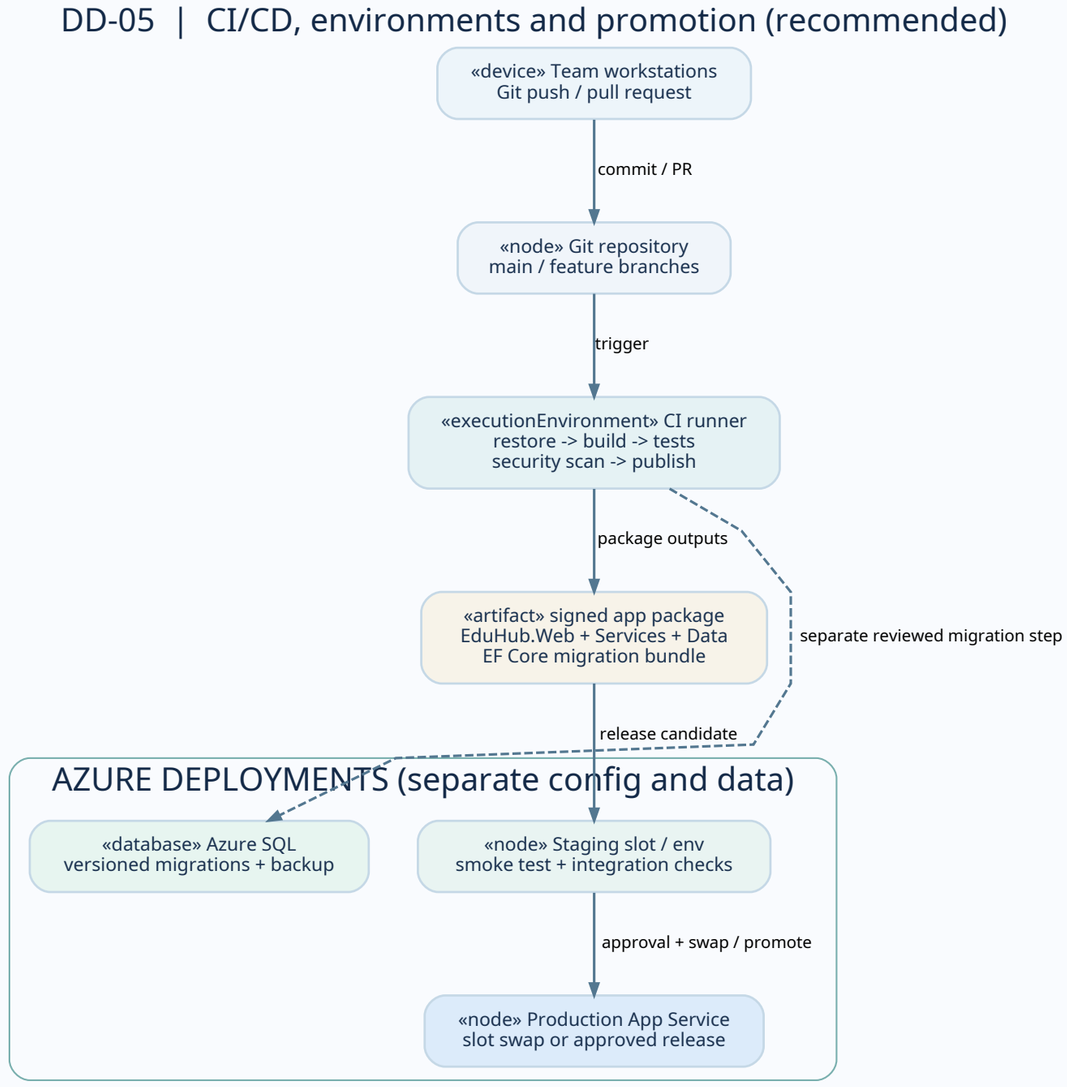
PAYMENT BOUNDARY CONTROLS

CONTROL	DESIGN REQUIREMENT	VERIFICATION
Public callback	Expose a stable HTTPS endpoint such as /webhooks/stripe.	Stripe test event reaches production/staging endpoint.
Raw-body signature	Verify Stripe-Signature with environment-specific signing secret.	Invalid or modified payload is rejected.
Idempotency	Persist unique external event ID before duplicating fulfillment.	Repeated events cannot create duplicate entitlements.
Order authority	Use verified session/payment state and stored price snapshot.	Client-supplied totals are not trusted.
Transaction boundary	Record payment settlement and enrollments consistently.	Failure before commit is recoverable; no partial grant.
Failure mode	Accept retries and non-guaranteed event ordering.	Out-of-order or duplicate delivery preserves correct state.
Secret storage	Stripe secret, webhook secret via Key Vault or protected app settings.	No live keys in source, browser or logs.

Stripe is a third-party system; the webhook is a synchronous inbound HTTP request, not a separately deployed EduHub service. A background queue can be added later for high-volume or long-running fulfillment, but is not required by the initial three-project architecture. [S1, R7]

11 | Release and environment topology

How one .NET build becomes a versioned Azure deployment.



READING THE DIAGRAM

A repository push triggers tests and creates a single application artifact plus versioned migration output. The recommended staging slot receives the release candidate; production promotion follows smoke tests and approval.

CAUTION: EF Core migrations must be reviewed and coordinated with production release.

12 | Environment and release strategy

Practical controls for a graduation project and an extendable production baseline.

PROPOSED ENVIRONMENT MATRIX

ENVIRONMENT	WEB RUNTIME	DATA / SECRETS	PURPOSE
Local development	ASP.NET Core MVC localhost	Local SQL / dev storage; Stripe test keys	Fast feature development and debugging.
Integration / staging	App Service staging slot or separate app	Separate non-production database and Blob account; test Stripe keys	Smoke and end-to-end verification.
Production	App Service production slot	Production Azure SQL / Blob; live Stripe secrets	Real users and protected data.

DEPLOYMENT SEQUENCE

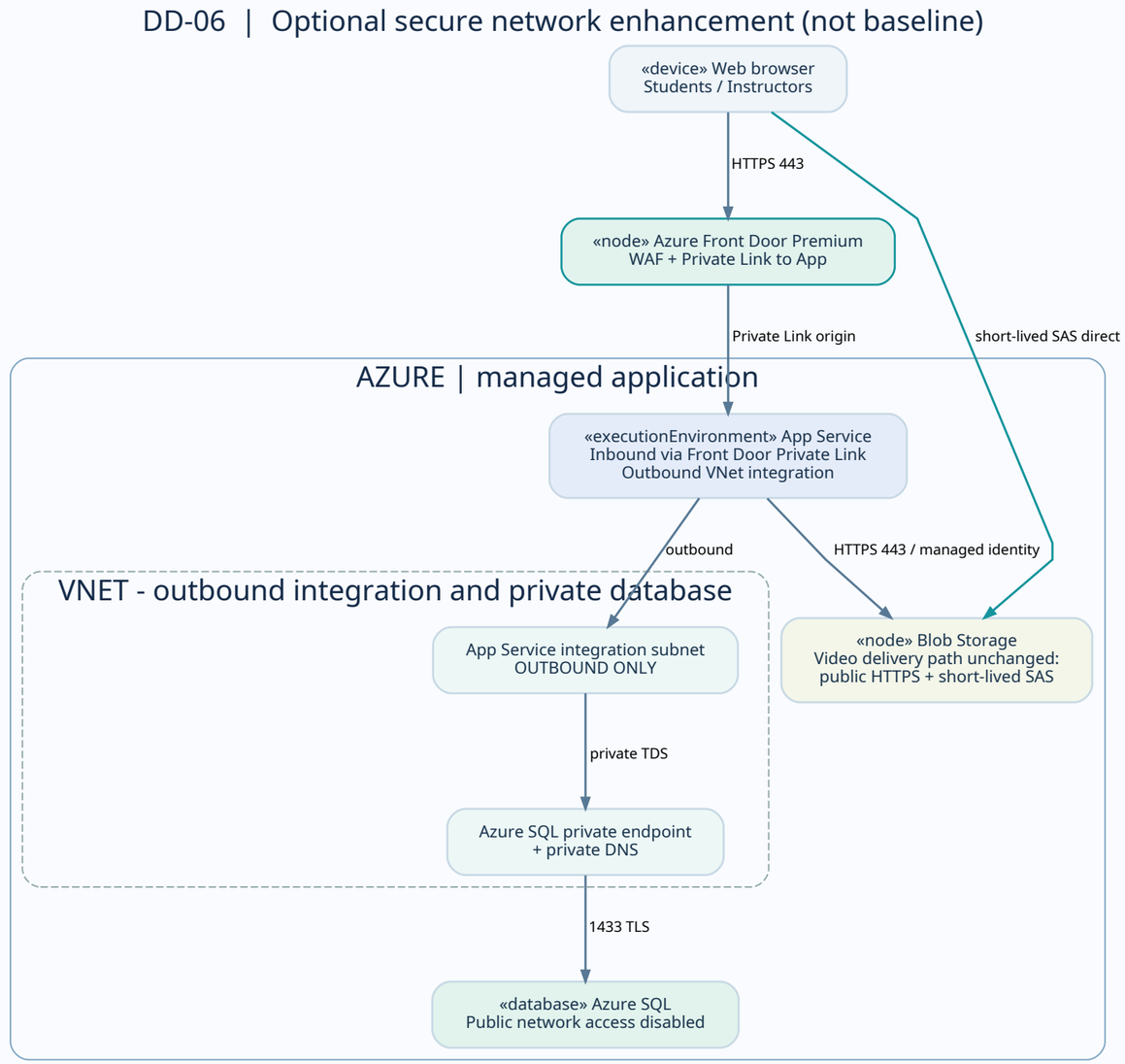
- Build gate. Restore dependencies, compile all three projects, run automated tests, static analysis and package EduHub.Web with referenced assemblies.
- Schema gate. Review the EF Core migration; back up production and apply controlled changes before (or compatibly with) release.
- Staging gate. Deploy to slot, run smoke tests for login, media, checkout sandbox and certificate generation; verify health telemetry.
- Promotion. Swap slots where supported, or deploy approved artifact to production; check health and payment-webhook reachability.
- Rollback. Swap back/restore artifact if compatible; database rollback requires an explicit, tested plan (schema changes may not reverse safely).

COST NOTE

Azure deployment slots require a qualifying App Service tier (Standard or above); a separate staging app is an alternative when slots are unavailable. [R5]

13 | Optional network hardening

A clearly separated enhancement that does not change the baseline media assumption.



READING THE DIAGRAM

Front Door Premium can privately connect to an App Service origin. App Service VNet integration covers outbound connectivity to a SQL private endpoint; it is not an inbound web security feature. Media retains browser-to-Blob SAS access here.

OPTIONAL: assess cost, DNS, private endpoints, SKU and operational complexity before approval.

14 | Security, observability and resilience

Operational controls shown or implied by the deployment topology.

DEPLOYMENT SECURITY CONTROL MATRIX

DOMAIN	RECOMMENDED CONTROL	OWNER / VALIDATION
Ingress	TLS-only, HTTPS redirect; origin restrictions when using Front Door; Premium managed WAF optional.	Azure platform + app config; direct-origin bypass test.
Identity	ASP.NET Core Identity roles; server-side object ownership checks.	Web and Services; role/entitlement tests.
Secrets	Managed identity to Key Vault and Azure services; environment-specific Stripe signing secrets.	Azure RBAC; credential-scan gate.
Database	Encrypted connection; restricted SQL endpoint; regular backups; least-privilege principal.	Azure SQL; restore drill.
Media	Private containers + constrained SAS; no long-lived signed URLs in logs.	Blob / Services; expired token test.
Payments	Signature verification, replay resistance, durable idempotency.	Webhook handler + SQL; repeated-event test.
Monitoring	Application Insights traces, dependency failures, exception rates and availability alerts.	Operations; synthetic login/health check.

RELIABILITY POSTURE

App Service is managed compute, but high availability is not guaranteed by a diagram alone. Production capacity, backup retention, restore RTO/RPO, slot count, Azure SQL service tier, storage redundancy and monitoring thresholds remain configuration decisions. Use test evidence to approve them. [R5, R8]

15 | Implementation configuration checklist

Actionable work items derived from the proposed physical topology.

CONFIGURATION VALIDATION

ID	DEPLOYMENT CHECK	PASS CONDITION
DP-01	Publish single MVC application with all three project assemblies.	App starts, routes render, DI resolves Services and Data.
DP-02	Create Azure SQL and migrate the defined identity/domain schema.	Migration succeeds; secret is not hard-coded.
DP-03	Configure private video containers with authorized access.	Anonymous GET fails; scoped authorized Range GET works.
DP-04	Connect Stripe Checkout and webhook endpoint.	Valid signed callback creates one entitlement exactly once.
DP-05	Set up HTTPS domain and optional Front Door edge.	Routing works; unauthorized origin bypass blocked when configured.
DP-06	Enable diagnostics, health checks and alert rules.	Synthetic failure appears in Insights; alert fires.
DP-07	Separate test/live secrets and data for each environment.	Test keys cannot charge live; staging cannot access production data.
DP-08	Define promotion and rollback gates.	Reproducible build; approved deploy; rollback drill documented.
DP-09	Confirm video processor design and package/license choices.	Media pipeline works; processing runtime explicitly approved.
DP-10	Run security tests on SAS expiry and payment replay.	Unauthorized playback/duplicate enrollment prevented.

16 | Traceability and architecture decisions

Preserve the requirements baseline and record additions without implying approval.

SOURCE REQUIREMENT -> DEPLOYED NODE

SOURCE REQUIREMENT	DEPLOYED EXECUTION / RESOURCE	DESIGN REFERENCE
ASP.NET Core MVC; Razor; Bootstrap 5	Azure App Service: EduHub.Web and static output	DD-01, DD-02
EF Core, Repository, Unit of Work, Identity	App Service: Services/Data; Azure SQL persistence	DD-01, DD-06
Azure Blob video storage; secure streaming	Blob Storage + authorized browser media path	DD-01, DD-03
Stripe paid courses and coupons	Stripe SaaS + MVC commerce, webhook + SQL	DD-01, DD-04
Certificates generated as PDF	App Service execution; output artifact in Blob	DD-01
Azure App Service + CDN delivery	App Service, proposed Front Door edge	DD-01, DD-02
Analytics dashboards; deployment	App Service MVC, SQL reports, Insights recommended	DD-01, DD-05

DECISIONS AWAITING APPROVAL

- ADR-DP01: Confirm the proposed Front Door Standard/Premium plan and whether CDN-backed video delivery is required in the first release.
- ADR-DP02: Confirm Azure SQL database tier and whether private SQL networking is in scope.
- ADR-DP03: Choose video transcoding/thumbnail processor and its compute location; no worker has been mandated.
- ADR-DP04: Confirm release environments, minimum App Service SKU and slot availability.
- ADR-DP05: Approve availability, backup/restore, alerting and security acceptance thresholds.

17 | References and UML review

Project-specific authority and current official Azure/Stripe engineering guidance.

SOURCE REFERENCE REGISTER

S1 EduHub - Online Learning Platform, 5-page uploaded specification.

S2 EduHub - Software Architecture and Technology Stack, previous approved deliverables.

R1 Microsoft Azure Architecture Center - N-tier architecture
https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/n-tier

R2 Microsoft - Azure Front Door / Azure CDN classic migration FAQ
https://learn.microsoft.com/en-us/azure/frontdoor/migration-faq

R3 Microsoft - Create a user delegation SAS for blobs in .NET
https://learn.microsoft.com/en-us/azure/storage/blobs/storage-blob-user-delegation-sas-create-dotnet

R4 Microsoft - App Service VNet integration
https://learn.microsoft.com/en-us/azure/app-service/overview-vnet-integration

R5 Microsoft - App Service deployment best practices
https://learn.microsoft.com/en-us/azure/app-service/deploy-best-practices

R6 Microsoft - Azure Front Door with Azure Storage blobs
https://learn.microsoft.com/en-us/azure/frontdoor/scenario-storage-blobs

R7 Stripe - Fulfill orders after successful Checkout
https://docs.stripe.com/checkout/fulfillment

R8 Microsoft - Application Insights introduction
https://learn.microsoft.com/en-us/azure/azure-monitor/app/app-insights-overview

R9 OMG - UML 2.5.1 specification
https://www.omg.org/spec/UML/2.5.1/

REVIEW STATUS

The primary deployment diagram uses UML deployment terminology for runtime nodes and artifacts. Supporting figures are focused topology and interaction views, intended to clarify media, payments, network boundaries and release operations. The topology is proposed; the PDF and editable source files do not assert that the infrastructure is already deployed.